StochLean

Klenke Chapters 01-08 - Work Handoff Design

Version 0.1

Primary coverage source: Achim Klenke, Probability Theory: A Comprehensive Course, 3rd ed. (2020)
Lean baseline for duplicate/API audit: Lean 4 / Mathlib v4.33.0
Scope: probability foundations (Ch.1-8) plus first stochastic-process consumers (Galton-Watson, Poisson process)

WORK HANDOFF SPECIFICATION - IMPLEMENTATION RULES ARE NORMATIVE

1. Purpose, Scope, and Normative Status

Purpose. This document is the implementation contract for the first StochLean foundation milestone. It preserves the approved Ch.1-8 design while removing discussion/history that Work does not need.

- Klenke Ch.1-8 is the mathematical coverage floor. Every formal Definition/Lemma/Theorem/Corollary is represented in the chapter ledger below.
- Coverage does not mean reimplementation. Mathlib and approved external code take precedence whenever semantics and usable API already exist.
- Lean files are organized by mathematical function and dependency, never by textbook chapter.
- The only mandatory new code is code that survives the duplicate audit and is required by a BUILD/BUILD-BRIDGE row or an AUDIT-FIRST row resolved to BUILD.
- Examples/exercises are implementation obligations only when explicitly marked load-bearing or acceptance-test material.

Status	Meaning	Required Work action
REUSE	Canonical implementation already exists.	Map to exact declaration/import; do not duplicate.
REUSE/BRIDGE	Core exists; thin reusable adapter may be needed.	Add only minimal wrapper/specialization with documented API gap.
BUILD	Confirmed foundation gap.	Implement rigorously, with no stronger/weaker semantic substitution.
BUILD/BRIDGE	A general result must be added, possibly by composing existing primitives.	Prefer the most generic useful theorem; avoid textbook-only APIs.
BUILD-CANDIDATE	Likely gap, but duplicate audit not yet closed.	Search first; implement only if no approved equivalent exists.
AUDIT-FIRST	Exact Mathlib/approved-external coverage is uncertain.	Resolve to REUSE/BRIDGE/BUILD before coding.
APPLICATION	Textbook application, not foundation API.	Record coverage; no blocking implementation.
DEFER	Valid material outside this milestone.	Record future destination; no current implementation.
EXTERNAL-CHECK	Textbook cites an external theorem or third-party code may exist.	Do not trust/import until independent audit is recorded.

2. Global Non-Negotiable Design Rules

Coverage rule: Klenke 3rd edition is the initial mathematical completeness baseline. Do not silently omit a labeled result. If a result is not implemented, its ledger status must explain why.

Functional organization rule: No Klenke/ChapterXX.lean layout. Put each object/theorem in one canonical functional module. Cross-chapter material has one implementation only.

No-duplicate rule: Before adding any public definition/theorem, search in this exact order: (1) pinned Mathlib revision; (2) the entire local library; (3) the approved-external ledger. An equivalent usable result forbids a parallel implementation.

No-error rule: A compiling proof is not sufficient. Do not weaken hypotheses, strengthen conclusions without proof, substitute a surrogate model, hide a conclusion as a structure field, or use vacuous/uninhabited assumptions to close a goal.

No-semantic-ambiguity rule: Explicitly distinguish pointwise vs a.e., same one-time law vs f.d.d. law vs path-space law, a.s. modification vs indistinguishability, weak convergence vs probability convergence, global vs local convergence in measure, and finite vs extended values.

No-totalization-abuse rule: If Mathlib totalizes an operation outside its natural mathematical domain (for example raw conditional expectation falling back to 0), public theorems must carry the genuine hypotheses or use typed domains. Never exploit fallback values as mathematics.

Proof-integrity rule: Project source must contain no sorry, admit, or project-defined axiom. Do not use conclusion-shaped assumptions. Public theorems must pass dependency/axiom inspection.

Canonical-representation rule: Use Mathlib-native objects whenever available: Measure/ProbabilityMeasure, Kernel, Lp, TendstoInMeasure, condExp/condDistrib, HasIndepIncrements, etc. Do not create a parallel StochasticProcess structure; processes remain X : T -> Omega -> E.

A.s. path rule: For uncountable time, an a.s. path property means one common full-measure event on which the entire trajectory has the property. Pairwise-a.s. statements are not substitutes.

Generic-upstream rule: If a result is not model-specific, place it in a generic namespace/module suitable for future Mathlib upstream. Do not lock generic probability theorems inside Galton-Watson/Poisson/PTASEP namespaces.

Source-correction rule: Known textbook errata/semantic defects listed in Section 7 override literal OCR/printed text. Work must implement the corrected mathematical statement.

Minimal-import rule: Final modules should use narrow imports. Avoid import Mathlib in production modules unless no maintainable alternative exists.

3. Duplicate, Trust-Boundary, and Provenance Rules

Layer	Trust status	Rule
Lean kernel + pinned Mathlib v4.33.0	Approved baseline	May be reused directly. Exact declaration and import path must be recorded for every REUSE row used by new code.
This StochLean repository	Approved after CI/audit	Search all modules before adding a duplicate. One canonical implementation per semantic object.
Third-party Lean libraries	Not approved by default	BrownianMotion, PFR, StatLean, formal-

Layer	Trust status	Rule
		mathfin, etc. are EXTERNAL-CHECK until a specific module/declaration is audited and entered in an approved-external ledger.
Klenke book	Coverage/mathematical source, not code dependency	Use as coverage floor and statement source. Textbook-cited external results remain EXTERNAL-CHECK/DEFER unless separately formalized or reused from approved code.
Adapted external code	Provenance required	Record repository, revision, source declaration, license, semantic changes, and local theorem mapping.

HARD GATE: a new public declaration may be merged only after the duplicate audit is recorded. "I did not know the theorem existed" is not an acceptable reason for duplication.

4. Work Execution Protocol, Acceptance, and Longevity

- Step A - Coverage audit: resolve every ledger row and every load-bearing exercise against the pinned Mathlib revision and approved-external ledger.
- Step B - API design: choose one canonical semantic representation; document natural domain and a.e./version issues before proving theorems.
- Step C - Implement only confirmed gaps. BRIDGE code must remain thin; REUSE rows should not spawn local copies.
- Step D - Semantic audit: test the edge cases listed in Section 8; verify hypotheses/conclusions against Klenke and corrected source notes.
- Step E - CI gate: lake build; warnings treated as errors for project code; no sorry/admit/project axiom; run lint where applicable; inspect axioms/dependencies of milestone theorems.
- Step F - Coverage gate: every BUILD resolved; every AUDIT-FIRST resolved; every EXTERNAL-CHECK either audited or explicitly deferred with no hidden dependency; exact source mapping recorded.

Maintenance trigger	Required action
Mathlib version upgrade	Re-run duplicate/API audit for all local public declarations touched by the upgrade; do not carry old REUSE/BUILD decisions forward blindly.
New external library approved	Re-run overlap search for affected modules; remove/deprecate local duplicates where the approved implementation becomes canonical.
Local generic theorem upstreamed to Mathlib	Migrate callers to Mathlib and retire the local copy after compatibility window.
Semantic definition changes	Bump design version and re-run downstream theorem audit; no silent API meaning change.
New Klenke correction/erratum discovered	Add source-correction entry, update ledger and affected formal statement before further dependent work.
A theorem requires a project-defined axiom/sorry to proceed	Stop. Mark the item blocked and return it for mathematical/design review; do not weaken the trust boundary.

5. Approved Functional Module Layout for Ch.01-08

```
Probability/
  GeneratingFunction/
    Basic.lean
    Analytic.lean
  Convergence/
    Discrete.lean
  LimitTheorems/
    PoissonApproximation.lean
    WeakLaw.lean                    # only if a real bridge remains after audit
  RandomSum.lean
  Distributions/
    Multinomial.lean                # only after duplicate/external audit
  Branching/
    GaltonWatson.lean
    Extinction.lean
  EmpiricalProcess/
    CDF.lean
    GlivenkoCantelli.lean
  MaximalInequality/
    Kolmogorov.lean                 # only if Mathlib specialization is insufficient
  Process/
    Path/
      Monotone.lean
      RightContinuous.lean
    StationaryIncrements.lean
    Poisson/
      Basic.lean
      IntervalCounts.lean
      UniformPoints.lean
      ArrivalTimes.lean
```

```
ForMathlib/
  MeasureTheory/
    Function/
      ConvergenceInMeasureLocal.lean
      UniformIntegrableEnvelope.lean
      DeLaValleePoussin.lean
      LpLocalConvergence.lean
  Probability/
    Conditional/
      Event.lean                    # only if a genuine bridge gap exists
      CondExpBridges.lean           # only if a genuine bridge gap exists
      CondDistribBridges.lean       # only if a genuine bridge gap exists
```

- This tree is a functional boundary, not a file-creation checklist. If duplicate audit closes a gap, omit the file.
- Do not add chapter-numbered namespaces/files. Do not create parallel definitions for convolution, kernels, Lp, conditional expectation, independent increments, or probability measures.
- Deferred domains (percolation, information theory, full concentration theory, Lp duality if absent, random measures/PPP, stochastic calculus) must not be pulled into this milestone unless required by a mandatory BUILD dependency.

6. Chapter 1 - Basic Measure Theory

Scope decision: Compatibility chapter. Reuse Mathlib foundations; add only genuine thin bridges. No new measure-theory foundation.

Klenke item(s)	Status	Work directive
1.1, 1.2, 1.3, 1.4, 1.6, 1.7, 1.8, 1.9, 1.10, 1.12, 1.13, 1.15, 1.16, 1.18, 1.19, 1.20, 1.21, 1.25, 1.26	REUSE	Set classes, sigma-algebras, generated structures, pi-lambda, topology/Borel, traces: map to Mathlib only.
1.23	REUSE/BRIDGE	Borel generators of R^n. Add only missing high-frequency generator wrappers.
1.27, 1.28, 1.29, 1.31, 1.33, 1.34, 1.35, 1.36, 1.38	REUSE	Set-function/measure continuity and probability-space basics. Do not rebuild content/premeasure hierarchy merely to match textbook notation.
1.41, 1.46, 1.47, 1.48, 1.49, 1.50, 1.51, 1.52, 1.53, 1.55	REUSE	Caratheodory/outer-measure/Lebesgue construction already belongs to Mathlib measure theory.
1.42	REUSE/BRIDGE	Uniqueness by pi-generator: use existing measure extensionality; thin wrapper only if downstream boilerplate remains.
1.57, 1.59, 1.60	REUSE	Lebesgue-Stieltjes/CDF correspondence. Use Mathlib CDF/StieltjesFunction API.
1.61, 1.64	REUSE	Finite/infinite Bernoulli product constructions: reuse product-measure infrastructure; Ch.14 later owns systematic product-space design.
1.65	AUDIT-FIRST	Measure approximation by generator/semiring. Resolve exact Mathlib coverage; bridge only if required by later proofs.
1.68, 1.69, 1.73	REUSE	Null sets, completion, restriction.
1.76, 1.78, 1.79, 1.80, 1.81, 1.82, 1.83, 1.84, 1.87, 1.88, 1.89, 1.90, 1.91, 1.92, 1.93, 1.95, 1.96, 1.98	REUSE	Measurability, generated sigma-algebras, products/coordinates, simple functions, image measures.
1.97	REUSE	Doob-Dynkin/factorization: use Mathlib FactorsThrough infrastructure.
1.101	REUSE	Change of variables: use Mathlib Jacobian/change-of-variables; Klenke cites external calculus source.
1.102, 1.103, 1.104, 1.106	REUSE	Random variables, laws, realization of CDFs, densities. Use Measure.map / IdentDistrib / CDF APIs.

Mandatory semantic/source notes

- Random variables remain measurable functions; do not define a RandomVariable structure.
- Standard distributions listed in Example 1.105/1.107 enter the availability ledger. Poisson, exponential, Gaussian are mandatory downstream dependencies; missing distributions may be deferred if not used.

7. Chapter 2 - Independence

Scope decision: Independence foundation is predominantly Mathlib reuse. Percolation remains an application/deferred domain.

Klenke item(s)	Status	Work directive
2.3, 2.5, 2.7, 2.11, 2.13	REUSE/BRIDGE	Independent events/classes, complement closure, Borel-Cantelli, generated-sigma independence. Use Mathlib Indep/iIndep; bridge only statement-shape gaps.
2.14, 2.16, 2.19, 2.20, 2.21, 2.22, 2.23, 2.26	REUSE/BRIDGE	Independent random variables, generator criteria, joint CDF/density criteria, existence of finite independent laws, regrouping by index blocks.
2.29, 2.31, 2.32	REUSE	Convolution and law of sums: use Mathlib measure convolution; never define a second convolution.
2.34, 2.35, 2.37, 2.38, 2.39	REUSE/BRIDGE	Tail sigma-algebra and Kolmogorov 0-1 law. Process-tail notation may wrap existing sigma-algebra machinery only.
2.40, 2.41, 2.42, 2.43, 2.44, 2.45, 2.47	APPLICATION	Percolation model/measurability/coupling/criticality/cluster uniqueness: record coverage, no current core implementation.
2.46	EXTERNAL-CHECK	Kesten theorem is cited, not proved by Klenke. Do not import into the foundation trust boundary.

Load-bearing examples/exercises

Source	Status	Work directive
Exercise 2.2.3 - multinomial law	BUILD-CANDIDATE	Load-bearing for Klenke 5.35 uniform-point Poisson construction. Audit Mathlib and approved external libraries first; implement distribution/API only if absent.

8. Chapter 3 - Generating Functions

Scope decision: First major BUILD chapter: law-first PGF, discrete convergence bridge, general Poisson approximation, random sums, Galton-Watson.

Klenke item(s)	Status	Work directive
3.1	BUILD	Canonical PGF is law-first on real z in [0,1] (unit interval subtype/domain). No divergent-series totalization.
3.2	BUILD	Continuity, smoothness on (0,1), factorial-moment boundary limits, uniqueness. Infinite boundary moments must use extended-value semantics where appropriate.
3.3	BUILD	Multiplicativity under independence; prove at law level and derive RV version through law bridge.
3.5	REUSE/BRIDGE	Generalized binomial theorem is analysis, not probability API. Use Mathlib if available; otherwise generic analysis bridge.
3.6	BUILD/BRIDGE	Discrete probability convergence equivalences: atomwise/setwise/PGF. Place in Probability/Convergence/Discrete.lean, not only PGF.
3.7	BUILD	Triangular-array Poisson approximation: sum p_nk -> lambda and sum p_nk^2 -> 0 implies weak convergence to Poi(lambda).
3.8	BUILD	Generic random-sum PGF composition. Put in Probability/RandomSum.lean.
3.9, 3.10, 3.11	BUILD	Canonical Galton-Watson construction, PGF iteration, extinction fixed-point theorem.

Mandatory semantic/source notes

- PGF Basic and Analytic must be separate layers. Basic domain is [0,1]; extension beyond 1 requires explicit convergence hypotheses.
- Theorem 3.11 must use the correct source assumption p_1 != 1. OCR may drop the negation; never copy OCR blindly.

9. Chapter 4 - The Integral

Scope decision: Coverage-only measure/integration chapter. Expected new core code: zero.

Klenke item(s)	Status	Work directive
4.1, 4.2, 4.3, 4.4, 4.6, 4.7, 4.8, 4.9, 4.10, 4.12, 4.13, 4.15, 4.16, 4.17, 4.19	REUSE	Simple-function construction, integral, image measure, densities, Lp basics: use Mathlib.
4.20, 4.21	REUSE	Monotone convergence and Fatou: use Mathlib.
4.23	REUSE	Riemann/Lebesgue compatibility: use existing integral infrastructure; no textbook-specific reproof.
4.26	REUSE	Layer-cake/tail integral. Use Mathlib Layercake.

Mandatory semantic/source notes

- If a Ch.4 file is created, the Work report must name the exact Mathlib API gap that required it.

10. Chapter 5 - Moments, LLN, and Poisson Process

Scope decision: Mixed reuse/build. Poisson process is the primary stochastic-process deliverable; Glivenko-Cantelli and random-sum bridges are mandatory generic additions.

Klenke item(s)	Status	Work directive
5.1, 5.3, 5.4, 5.6, 5.7, 5.8	REUSE	Moments, expectation, variance/covariance, Cauchy-Schwarz, independence implies uncorrelated.
5.5, 5.10	BUILD-CANDIDATE	Wald and Blackwell-Girshick random-sum identities. Audit exact Mathlib coverage; if absent implement generically in RandomSum.lean.
5.11	REUSE	Markov/Chebyshev inequalities.
5.12	REUSE	Do not create parallel WLLN/SLLN predicates unless a reusable property wrapper is later justified. Use canonical convergence notions.
5.14	REUSE/BRIDGE	Quantitative WLLN for uncorrelated bounded-variance sequence; thin specialization from variance + Chebyshev.
5.16, 5.17, 5.18, 5.19, 5.20	REUSE	Strong law/Etemadi and proof internals. Use Mathlib StrongLaw; do not reproduce textbook truncation proof merely for coverage.
5.22	BUILD	Empirical CDF core definition/API.
5.23	BUILD	Full Glivenko-Cantelli for arbitrary real CDF, including discontinuous laws. No atomless/continuous-CDF weakening.
5.25, 5.26, 5.27	DEFER	Entropy/source coding belongs to InformationTheory, not stochastic-process foundation.
5.28	AUDIT-FIRST	Kolmogorov maximal inequality. Prefer specialization of existing martingale/maximal tools if semantically exact; build only missing functionality.
5.29	BUILD	Rate consequence from Kolmogorov inequality + Borel-Cantelli; lower priority than Poisson/GC.
5.30, 5.32	EXTERNAL-CHECK	Rademacher-Menshov and Baum-Katz are cited by Klenke, not proved; defer unless independently formalized.
5.33	BUILD	Poisson process definition, using author-corrected path regularity. Reuse Mathlib HasIndepIncrements.
5.34	BUILD	Equivalence between local interval-count axioms P1-P5 and Poisson process characterization.
5.35	BUILD	Uniform-point construction on [0,1]; prove joint disjoint-increment law via multinomial.
5.36	BUILD	Exponential-arrival construction on [0,infinity); formal proof must handle arbitrary finite ordered partition, not only n=2.

Load-bearing examples/exercises

Source	Status	Work directive
Exercise 5.1.1 - Paley-Zygmund	BUILD-CANDIDATE	Useful generic probability inequality; audit before build.
Exercise 5.2.1 - Bernstein-Chernoff	DEFER	Concentration-theory extension; not a Ch.01-08 blocker.
Exercise 5.5.1	APPLICATION	Use as construction/acceptance test for extension of uniform-point process to successive unit intervals.

Mandatory semantic/source notes

- Definition 5.33 must include the official correction: almost surely t -> N_t is monotone increasing and right-continuous.
- Nat.sub truncation must never be used as a substitute for path monotonicity. The path property is a separate a.s. trajectory statement.
- Create generic HasStationaryIncrements only if duplicate audit confirms Mathlib lacks it; design it for reuse by Poisson, Brownian, and later Levy processes.

11. Chapter 6 - Convergence Theorems

Scope decision: Major compatibility-extension chapter. Klenke uses local convergence in measure and a stronger sigma-finite uniform-integrability notion; do not conflate them with Mathlib global notions.

Klenke item(s)	Status	Work directive
6.1	REUSE	Measurability of distance between measurable maps.
6.2	BUILD	Define TendstoLocallyInMeasure using all finite-measure restrictions. Prove finite-measure equivalence with Mathlib TendstoInMeasure.
6.7	BUILD/BRIDGE	Exhaustion-based integral pseudometric characterizes local convergence. On raw functions call it a pseudometric; metric only after a.e. quotient.
6.8	REUSE	L1/mean convergence.
6.12	BUILD	Fast-convergence/Borel-Cantelli criteria for the local notion.
6.13	BUILD	Subsequence criterion: local convergence in measure iff every subsequence has an a.e.-convergent subsubsequence.
6.14	DEFER	Non-topologizability of a.e. convergence is coverage-only.
6.15	BUILD	Completeness/Cauchy convergence for local convergence in measure with complete target.
6.16	BUILD	Define a separately named sigma-finite envelope notion, e.g. UniformIntegrableByEnvelope. Do not shadow Mathlib UniformIntegrable.
6.17	BUILD	Equivalent envelope/tail formulations; prove finite-measure compatibility with Mathlib UI.
6.18	BUILD/BRIDGE	Closure under finite families, add/sub/abs, domination.
6.19	BUILD	de la Vallee-Poussin criterion if duplicate audit confirms absent. Prefer statement over canonical Mathlib UI on finite measure.
6.20	REUSE	Lp boundedness.
6.21, 6.22	REUSE/BRIDGE	Lp-bounded implies UI; bounded mean/variance specialization.
6.23	REUSE/BRIDGE	Positive integrable density on sigma-finite space.
6.24	BUILD	Klenke envelope UI vs uniform absolute continuity / finite equivalent-measure form.
6.25	BUILD	Exact sigma-finite Vitali: local convergence in measure + envelope UI iff L1 convergence.
6.26	REUSE	Dominated convergence.
6.27, 6.28	REUSE/BRIDGE	Continuity/differentiation under integral via Mathlib ParametricIntegral; only probability-specialized wrappers if repeatedly needed.

Load-bearing examples/exercises

Source	Status	Work directive
Exercise 6.1.3 - Egorov	REUSE	Use Mathlib Egorov.
Exercise 6.2.1	BUILD/BRIDGE	Same pseudometric-vs-metric safeguard as 6.7; quotient by a.e. for a genuine metric.

Mandatory semantic/source notes

- Klenke 6.2 is local convergence in measure on sigma-finite spaces; Mathlib TendstoInMeasure is global. They coincide on finite/probability spaces only.
- Theorem 6.7 is not a genuine metric on raw measurable functions: null modifications have distance zero. Formalize a pseudometric or an a.e.-quotient metric.
- Theorem 6.28 statement has a domain typo in the book: the differentiation conclusion is for x in the open interval I, not x in E.

12. Chapter 7 - Lp Spaces and Radon-Nikodym

Scope decision: Predominantly mature Mathlib analysis. Add only the local-convergence characterization that depends on the Ch.6 extensions.

Klenke item(s)	Status	Work directive
7.1, 7.2	REUSE	Use Mathlib Lp quotient/norm and Lp convergence.
7.3	BUILD/BRIDGE	Reuse Lp completeness; add only Klenke-style characterization using local convergence in measure + envelope UI for |f_n|^p.
7.4, 7.6, 7.7, 7.8, 7.9, 7.10, 7.11, 7.15, 7.16, 7.17, 7.18	REUSE	Convexity/Jensen/Young/Holder/Minkowski/ Fischer-Riesz.
7.19, 7.20, 7.21, 7.22, 7.23, 7.24, 7.25, 7.26, 7.27, 7.28	REUSE	Inner-product/Hilbert/orthogonal decomposition/Riesz-Frechet.
7.29, 7.30, 7.33, 7.34	REUSE	Density uniqueness, absolute continuity/singularity, Lebesgue decomposition, Radon-Nikodym.
7.35, 7.37	REUSE/BRIDGE	Total continuity/absolute-continuity formulations: map to Mathlib measure absolute-continuity APIs; no parallel notion unless exact semantic gap is proven.
7.40, 7.43, 7.44, 7.45, 7.46	REUSE	Signed measures, Hahn/Jordan, variation consequences.
7.47, 7.49, 7.50	AUDIT-FIRST	Lp dual-space theorem. If absent, defer to general functional-analysis/ForMathlib work; do not block stochastic-process foundation.

Mandatory semantic/source notes

- Theorem 7.25 has an author erratum affecting a sign in the proof. Use the correct Mathlib theorem rather than reproducing the text proof.
- Ch.7 should normally create no chapter-specific module. New code belongs only to generic ForMathlib functionality (not textbook namespace).

13. Chapter 8 - Conditional Expectations

Scope decision: Mostly direct Mathlib reuse. Primary work is semantic guarding and thin process-friendly bridges; do not rebuild conditional expectation or kernels.

Klenke item(s)	Status	Work directive
8.2	REUSE/BRIDGE	Event conditioning. Public API should require P(B)>0 (or a typed conditional measure); do not treat the book's arbitrary zero-mass value as canonical mathematics.
8.4, 8.5, 8.6, 8.7	REUSE/BRIDGE	Conditional measure, independence criterion, total probability, Bayes.
8.9, 8.10	REUSE/BRIDGE	Conditional expectation on a countable partition: derive from general condExp; no second CE definition.
8.11, 8.12, 8.13, 8.14	REUSE	General conditional expectation, existence/uniqueness, conditioning on a random variable, algebra/tower/pull-out/independence/dominated convergence.
8.17	REUSE	L2 orthogonal-projection characterization.
8.20, 8.21, 8.22	REUSE	Conditional Jensen, Lp contraction, uniform integrability of conditional expectations.
8.24	REUSE / NO DIRECT POINTWISE API	Conditioning on X=x is version-dependent. Expose kernels/functions only with law-a.e. uniqueness; do not create a globally canonical scalar condExpectationAt.
8.25	REUSE	Use Mathlib Kernel / MarkovKernel. Never define TransitionKernel in parallel.
8.28, 8.29	REUSE	Regular conditional distribution; use Mathlib condDistrib / StandardBorel infrastructure.
8.34, 8.35	REUSE	Measurable isomorphism / Klenke “Borel space” maps to Mathlib StandardBorelSpace, not a new BorelSpace-like class.
8.36	REUSE	Polish Borel space -> standard Borel: use existing topology/measurable-space theorem. Klenke cites this topological result.
8.37, 8.38	REUSE	Existence of RCD on standard Borel target and CE as integral against the RCD kernel.

Load-bearing examples/exercises

Source	Status	Work directive
Exercise 8.2.5 - conditional Markov inequality	REUSE/BRIDGE	Add a thin theorem only if exact Mathlib conditional inequality is inconvenient/missing.
Exercise 8.2.6 - conditional Cauchy-Schwarz	REUSE/BRIDGE	Same rule; prefer CondJensen/conditional Lp API.

Mandatory semantic/source notes

- Raw Mathlib conditional expectation may be totalized outside natural hypotheses. Public theorems must carry the genuine sub-sigma/integrability/sigma-finiteness conditions or operate on typed L1/L2 objects.
- Pointwise conditional probabilities P(A | X=x) are only version-defined P_X-a.e. in general; uniqueness statements must be law-a.e.
- Klenke “Borel space” is the standard Borel notion. Use StandardBorelSpace.
- Remark 8.26 has an author correction concerning sufficient generator/finiteness conditions. Do not reproduce that proof; rely on Mathlib Kernel measurability infrastructure.

14. Source Corrections and Semantic Regression Tests

Source issue	Normative implementation requirement
Klenke 3.11 OCR may read p1 = 1	Use actual theorem assumption p1 != 1.
Klenke 5.33 printed definition omits condition (iii)	Use author-corrected Poisson process: a.s. monotone increasing and right-continuous paths, in addition to N0=0 and the increment conditions.
Klenke 5.36 writes detailed proof only for n=2	Lean theorem must prove independence/correct laws for every finite ordered partition.
Klenke 6.7 calls d~ a metric on raw functions	Implement pseudometric semantics on raw functions; genuine metric only after quotient by a.e. equality.
Klenke Ex.6.2.1 same issue	Same safeguard for d_H.
Klenke 6.28 domain typo	Conclusion quantified over x in I.
Klenke 7.25 proof sign erratum	Use corrected statement/proof from canonical Mathlib result.
Klenke 8.2/8.9 zero-probability convention	Do not expose arbitrary 0 fallback as canonical conditional probability/expectation.
Klenke 8.24 P(. | X=x)	Treat as a kernel/version with P_X-a.e. uniqueness, not a pointwise globally unique object.
Klenke 8.35 terminology	Map “Borel space” to StandardBorelSpace.

Source issue	Normative implementation requirement
Klenke 8.26 generator remark erratum	Use canonical Kernel measurability API; do not reproduce incomplete argument.

Mandatory regression/semantic tests

- Local-vs-global convergence: on Lebesgue R, f_n = 1_[n,n+1] must converge locally in measure to 0 but not in global Mathlib TendstoInMeasure.
- Null modification: f=0 and g=1_{0} under Lebesgue measure must have zero exhaustion pseudodistance while remaining unequal as raw functions.
- PGF domain: a heavy-tail law whose power series diverges for every z>1 must still have a valid basic PGF on [0,1]; no totalized out-of-domain theorem.
- Glivenko-Cantelli: include a discrete/jump CDF test, not only continuous laws.
- Poisson Nat subtraction: verify path monotonicity is an explicit hypothesis/property and is not inferred from truncated subtraction.
- A.s. path property: test that the definition quantifies one full-measure trajectory event, not one null set per time pair.
- Poisson 5.36: compile a finite partition with at least four increments as a theorem/application test.
- Conditional expectation: invalid-domain totalization must not make public theorems provable without integrability/sub-sigma hypotheses.
- Conditioning at a point: two versions differing on a P_X-null set must be recognized as equivalent for the intended API.

15. Work Implementation Order and Milestone Completion Definition

Phase	Required deliverable
A	Full duplicate/API audit on pinned Mathlib and local tree; resolve AUDIT-FIRST/BUILD-CANDIDATE rows; create exact declaration map for REUSE rows touched by later code.
B	PGF Basic + Analytic; discrete convergence bridge; triangular-array Poisson approximation.
C	RandomSum; Galton-Watson construction/PGF/extinction; resolve multinomial dependency.
D	Empirical CDF + full arbitrary-CDF Glivenko-Cantelli.
E	Random-sum moment identities and maximal inequality/rate only after exact coverage audit.
F	Process path predicates + stationary increments gap (if confirmed); corrected Poisson process, P1-P5 characterization, both constructions.
G	Ch.6 local convergence + envelope UI + de la Vallee-Poussin + sigma-finite Vitali; Lp local characterization.
H	Conditional-event/condExp/condDistrib bridges only where a real API gap survives; otherwise REUSE-only mapping.
I	Final coverage, duplicate, semantic, source-correction, dependency/axiom, and CI audit.

MILESTONE COMPLETE ONLY WHEN: every BUILD is proved; every AUDIT-FIRST and BUILD-CANDIDATE is resolved; every EXTERNAL-CHECK is either audited or explicitly deferred with no hidden dependency; the Ch.1-8 ledger has 100% disposition; and all hard semantic regression tests pass.

16. Expected Net-New Core After Duplicate Audit

- Probability generating function API with safe domain + analytic layer.
- Discrete probability convergence bridge and Klenke triangular-array Poisson approximation.
- Generic random-sum API; Galton-Watson process and extinction theorem.
- Empirical CDF and full Glivenko-Cantelli theorem.
- Poisson process foundation: common-a.s. monotone/right-continuous paths, stationary increments if absent, P1-P5 characterization, uniform-point and exponential-arrival constructions.
- Local convergence in measure on sigma-finite spaces, with finite-space compatibility to Mathlib.
- Klenke envelope uniform integrability, de la Vallee-Poussin criterion if absent, sigma-finite Vitali theorem, and Lp local-convergence characterization.
- Only thin conditional-probability/conditional-expectation/kernel bridges that survive duplicate audit.

17. Mathlib Anchor Modules Already Identified

Area	Canonical Mathlib anchors (non-exhaustive)
CDF / law uniqueness	Mathlib/Probability/CDF.lean
Borel-Cantelli / zero-one	Mathlib/Probability/BorelCantelli.lean; Probability/Independence/ZeroOne.lean
Strong law	Mathlib/Probability/StrongLaw.lean
Convergence in measure	Mathlib/MeasureTheory/Function/ConvergenceInMeasure.lean
Uniform integrability / Vitali	Mathlib/MeasureTheory/Function/UniformIntegrable.lean
Parametric integral differentiation	Mathlib/Analysis/Calculus/ParametricIntegral.lean
Holder/Lp	Mathlib/MeasureTheory/Function/Holder.lean and LpSpace modules
Radon-Nikodym / signed measures	MeasureTheory/VectorMeasure/Decomposition/RadonNikodym.lean; Hahn/Jordan modules
Conditional expectation	MeasureTheory/Function/ConditionalExpectation/ {Basic,Real,PullOut,CondJensen,CondexpL1,CondexpL2}.lean

Area	Canonical Mathlib anchors (non-exhaustive)
Regular conditional distribution	Mathlib/Probability/Kernel/CondDistrib.lean + StandardBorel disintegration modules
Independent increments	Mathlib/Probability/Independence/Process/HasIndepIncrements/Basic.lean
Layer cake	Mathlib/MeasureTheory/Integral/Layercake.lean
Factorization	Mathlib/MeasureTheory/Function/FactorsThrough.lean

Note: This anchor table is not a substitute for the required exact duplicate audit at implementation time. Mathlib upgrades may change file names, theorem names, or available generality.

18. Source Baseline

- Achim Klenke, Probability Theory: A Comprehensive Course, Third Edition, Springer, 2020. User-provided complete PDF is the primary Ch.1-8 coverage source.
- Author errata/corrections already incorporated into this approved design where explicitly listed in Section 14.
- Lean/Mathlib implementation baseline: Lean 4 / Mathlib v4.33.0 for the initial Work audit. Any version change triggers the maintenance rule in Section 4.